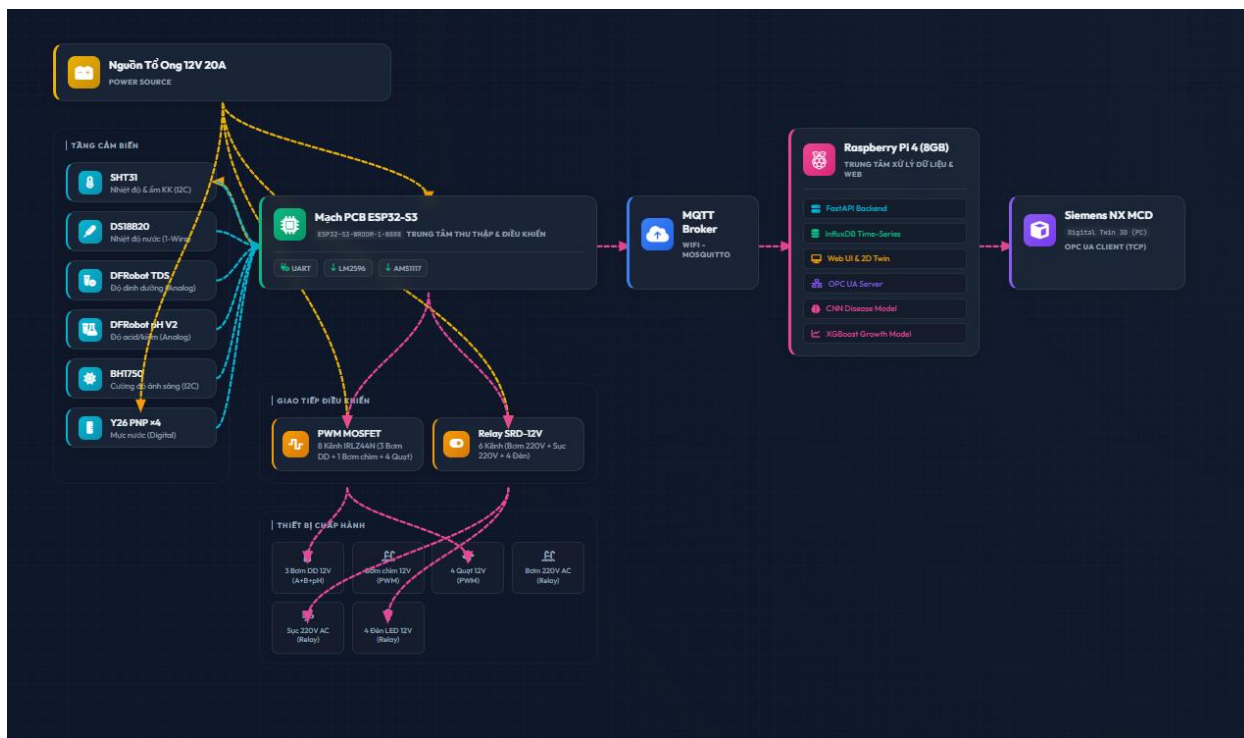


Sơ đồ khối hệ thống



1. Giới thiệu ngắn gọn về FastAPI

FastAPI là một framework Python hiện đại, hiệu năng cao dùng để xây dựng các API. Framework này nổi bật với tốc độ xử lý nhanh (tương đương NodeJS và Go) nhờ hỗ trợ lập trình bất đồng bộ (`async/await`), tự động tạo tài liệu kiểm thử API (Swagger UI), và kiểm soát kiểu dữ liệu đầu vào cực kỳ chặt chẽ thông qua Pydantic.

2. Các đầu mục công việc chính sẽ thực hiện với FastAPI

Thu thập và quản lý dữ liệu cảm biến: Nhận dữ liệu từ ESP32 qua MQTT và lưu trữ vào InfluxDB.

Cung cấp API điều khiển thiết bị: Nhận lệnh từ giao diện Web UI để điều chỉnh quạt (PWM) và relay (bơm, đèn).

Tích hợp và vận hành các mô hình AI: Chạy mô hình CNN (TFLite) nhận diện bệnh lá và XGBoost dự báo sinh trưởng của cây.

Đồng bộ hóa thời gian thực (Real-time): Dùng WebSocket để đẩy dữ liệu lập tức lên màn hình giám sát Web UI.

Liên kết Digital Twin 3D qua OPC UA: Ghi dữ liệu cảm biến và trạng thái thiết bị lên OPC UA Server để đồng bộ với mô hình mô phỏng Siemens NX MCD.

3. Chi tiết từng nội dung thực hiện

A. Thu thập dữ liệu và Điều khiển thiết bị (REST API & MQTT)

Thu thập dữ liệu: FastAPI lắng nghe luồng dữ liệu gửi từ ESP32 gửi lên qua MQTT Broker. Mỗi khi có bản tin JSON mới (gồm nhiệt độ, độ ẩm, pH, TDS, mực nước), FastAPI sẽ bóc tách và ghi thẳng vào InfluxDB phục vụ vẽ biểu đồ lịch sử.

Các Endpoint điều khiển:

POST /api/control/pwm: Nhận giá trị duty cycle (0-255) từ Web UI để điều khiển tốc độ 4 quạt thông gió và 3 bơm nhu động châm dinh dưỡng.

POST /api/control/relay: Nhận lệnh bật/tắt (True/False) cho bơm chính 220V, máy sục khí và 4 cụm đèn LED Grow.

Các Endpoint truy xuất dữ liệu:

GET /api/sensor-data/latest: Lấy nhanh trạng thái cảm biến mới nhất.

GET /api/sensor-data/history?hours=X: Truy xuất dữ liệu lịch sử để hiển thị biểu đồ.

B. Triển khai AI trực tiếp trên Backend

Nhận diện bệnh hại (CNN MobileNetV2 - TFLite):

Xây dựng Endpoint **POST /api/ai/disease-detect** nhận file ảnh tải lên từ Web App.

Sử dụng thư viện TensorFlow Lite interpreter trên Pi để xử lý ảnh (chuyển đổi về ma trận $224 \times 224 \times 3$), chạy mô hình nhận diện và trả về kết quả phân loại bệnh (Ví dụ: Khỏe mạnh, Thối lá, Cháy lá...) cùng độ tin cậy.

Dự đoán sinh trưởng (XGBoost):

Xây dựng Endpoint **GET /api/ai/growth-predict** chạy định kỳ.

Sử dụng thư viện XGBoost để dự đoán tiến độ phát triển của cây (**growth_percent** từ 0% - 100%) và ước tính số ngày còn lại đến kỳ thu hoạch (**days_remaining**) từ dữ liệu môi trường tích lũy.

C. Giao tiếp thời gian thực & Đồng bộ Digital Twin

WebSocket Server: FastAPI mở một cổng kết nối WebSocket giúp Web App Dashboard luôn cập nhật thông số cảm biến theo thời gian thực mà không cần tải lại trang.

Cầu nối OPC UA (Siemens NX MCD):

FastAPI kết nối trực tiếp với Python OPC UA Server (**asynucua**).

Mỗi khi nhận thông tin mới từ cảm biến hoặc lệnh điều khiển từ Web UI, FastAPI sẽ cập nhật giá trị vào các Node tương ứng trên OPC UA Server.

Phần mềm Siemens NX MCD kết nối vào OPC UA Server này để điều khiển mô hình cơ điện tử 3D (xoay cánh quạt, kích hoạt dòng chảy chất lỏng của bơm, bật đèn LED sáng đỏ, co giãn kích thước cây ảo theo tỉ lệ phần trăm phát triển thực tế).

1. Giới thiệu ngắn gọn về InfluxDB

InfluxDB là một cơ sở dữ liệu mã nguồn mở chuyên dụng cho dữ liệu chuỗi thời gian (Time-Series Database). Nó được tối ưu hóa để lưu trữ và truy vấn tốc độ cao các dữ liệu có gắn mốc thời gian (timestamp) liên tục, cực kỳ thích hợp cho các hệ thống giám sát IoT nơi dữ liệu cảm biến được gửi về dồn dập theo từng giây hoặc từng phút.

2. Các đầu mục công việc chính thực hiện với InfluxDB

Lưu trữ dữ liệu cảm biến liên tục: Ghi lại toàn bộ lịch sử thông số môi trường (nhiệt độ, độ ẩm, pH, TDS...) theo thời gian thực.

Ghi nhận trạng thái thiết bị: Lưu lịch sử bật/tắt của các relay và mức công suất của các công điều khiển quạt/bơm (PWM).

Lưu kết quả dự đoán từ AI: Lưu trữ kết quả phân tích bệnh cây từ mô hình CNN và tiến độ phát triển rau từ XGBoost.

Cung cấp dữ liệu lịch sử cho Web UI: Hỗ trợ FastAPI truy vấn nhanh dữ liệu quá khứ để vẽ biểu đồ trực quan hóa.

3. Chi tiết từng nội dung thực hiện

A. Cấu trúc lưu trữ dữ liệu (Schema Design)

Dữ liệu trong InfluxDB được tổ chức dưới dạng các Measurements (tương tự như bảng trong SQL) với cấu trúc tối ưu:

Measurement sensor_data (Dữ liệu cảm biến):

Fields (Giá trị số liệu): air_temp (float), air_humidity (float), water_temp (float), tds (float), ph (float), light_intensity (float).

Tags (Nhãn tìm kiếm nhanh): device_id="esp32_01", location="tầng_1".

Measurement actuator_status (Trạng thái thiết bị):

Fields: pump_dd_1_pwm (int), pump_dd_2_pwm (int), fan_1_pwm (int), pump_220v (boolean), grow_light_1 (boolean)...

Tags: device_id="esp32_01".

Measurement ai_predictions (Kết quả từ AI):

Fields: disease_label (string - tên bệnh chẩn đoán), confidence (float), growth_percent (float), days_remaining (float).

B. Cơ chế ghi dữ liệu (Data Ingestion)

Tần suất ghi: ESP32-S3 gửi dữ liệu lên mỗi 30 giây qua MQTT. FastAPI đón nhận dữ liệu này, đóng gói thành các điểm dữ liệu (Points) và sử dụng thư viện influxdb-client-python ghi bất đồng bộ (async) vào InfluxDB.

Cài đặt chạy ngầm (Systemd Service): Trên Raspberry Pi 4, InfluxDB được cài đặt cấu hình tự khởi động cùng hệ thống (systemd) để đảm bảo dịch vụ ghi dữ liệu không bị ngắt quãng kể cả khi Pi bị mất nguồn đột ngột.

C. Cơ chế truy vấn phục vụ hiển thị biểu đồ

Sử dụng ngôn ngữ Flux: FastAPI sẽ gọi các câu lệnh truy vấn bằng ngôn ngữ Flux để lấy dữ liệu. Ví dụ: Lấy trung bình thông số cảm biến theo từng giờ trong vòng 24 giờ qua (aggregateWindow(every: 1h, fn: mean)).

Tối ưu hóa bộ nhớ: InfluxDB tự động quản lý việc dọn dẹp hoặc nén dữ liệu cũ (Retention Policies) để tránh làm đầy dung lượng thẻ nhớ của Raspberry Pi 4 sau thời gian dài vận hành.

1. Giới thiệu ngắn gọn về Web UI (Giao diện người dùng)

Web UI là bảng điều khiển và giám sát trung tâm (Dashboard), giúp người dùng tương tác trực quan với hệ thống trồng cây thủy canh. Giao diện được xây dựng bằng ngôn ngữ web tiêu chuẩn (HTML/CSS/JS) nhằm mục tiêu hiển thị số liệu cảm biến tức thời, hỗ trợ điều khiển thiết bị từ xa, và thực hiện tải ảnh lên để chẩn đoán bệnh bằng AI.

2. Các đầu mục công việc chính thực hiện với Web UI

Hiển thị thông số giám sát thời gian thực: Trực quan hóa dữ liệu cảm biến (Nhiệt độ, pH, TDS, độ ẩm...) và trạng thái thiết bị.

Tích hợp mô hình Digital Twin 2D: Sử dụng đồ họa Canvas/SVG động để hiển thị trạng thái chuyển động của dòng nước, quạt và đèn ảo ngay trên trang web.

Bảng điều khiển thiết bị ngoại vi: Cung cấp các thanh trượt (slider) điều chỉnh quạt/bơm (PWM) và nút gạt (toggle switch) bật/tắt relay.

Chẩn đoán bệnh cây bằng AI thủ công: Tích hợp tính năng chụp ảnh trực tiếp từ camera điện thoại hoặc tải ảnh lên để phân tích bệnh lá.

Biểu đồ phân tích lịch sử: Vẽ đồ thị diễn biến các chỉ số môi trường bằng thư viện Chart.js.

3. Chi tiết từng nội dung thực hiện

A. Giám sát & Điều khiển thời gian thực (Real-time Dashboard)

Nhận dữ liệu từ WebSocket: Trang Web thiết lập kết nối WebSocket tới FastAPI. Khi có dữ liệu mới đẩy về, giao diện sẽ tự động cập nhật các con số và thay đổi màu sắc chỉ báo mức nước mà không cần tải lại trang.

Giao diện điều khiển:

Thanh trượt (Slider): Cho phép kéo chọn từ 0 đến 255 để tăng giảm tốc độ quạt thông gió hoặc lưu lượng bơm dinh dưỡng. Sự thay đổi sẽ gửi ngay request POST /api/control/pwm tới Backend.

Nút gạt (Toggle Switch): Điều khiển Bật/Tắt các Relay của đèn LED Grow, bơm tuần hoàn chính và bơm sục khí.

B. Mô phỏng Digital Twin 2D

Thiết kế một sơ đồ hệ thống giàn cây NFT trực quan bằng đồ họa 2D (Canvas hoặc SVG động).

Hiệu ứng động:

Khi bật quạt trên thực tế hoặc mô phỏng, cánh quạt trên bản vẽ 2D của Web UI sẽ xoay tương ứng.

Khi bật máy bơm, đường ống nước ảo trên màn hình sẽ có hiệu ứng dòng chảy chuyển động để người dùng dễ hình dung trạng thái hoạt động trực quan.

C. Chẩn đoán và Dự đoán từ AI

Chức năng chẩn đoán lá bệnh:

Giao diện tích hợp thẻ tải ảnh sử dụng thuộc tính HTML5 capture="camera". Khi người dùng dùng điện thoại mở web, nhấn vào nút chụp ảnh, camera điện thoại sẽ tự động mở lên để chụp cận cảnh chiếc lá nghi ngờ bị bệnh.

Ảnh chụp được gửi dưới dạng Multi-part form-data lên endpoint /api/ai/disease-detect của FastAPI và hiển thị kết quả chẩn đoán trả về trực tiếp trên màn hình kèm tỉ lệ phần trăm chính xác.

Hiển thị dự báo sinh trưởng:

Hiện thị chỉ số phần trăm trưởng thành của cây và đồng hồ đếm ngược số ngày còn lại đến kỳ thu hoạch được tính toán từ mô hình XGBoost.

D. Thư viện biểu đồ (Chart.js)

Tích hợp thư viện Chart.js để vẽ các đồ thị lịch sử nhiệt độ, pH, TDS trong 24 giờ qua hoặc theo tuần, giúp người dùng nắm được xu hướng biến động dinh dưỡng trong bể chứa nhằm đưa ra điều chỉnh hợp lý.

1. Giới thiệu ngắn gọn về OPC UA

OPC UA (Open Platform Communications Unified Architecture) là giao thức truyền thông công nghiệp chuẩn hóa, độc lập với nền tảng, giúp các thiết bị phần cứng, PLC và các phần mềm mô phỏng (như Siemens NX) trao đổi dữ liệu an toàn, tin cậy. Trong hệ thống này, nó đóng vai trò là "cầu nối" và "bộ nhớ chung" liên kết giữa mô hình 3D ảo và thuật toán điều khiển phần mềm.

2. Nguyên lý hoạt động và Sơ đồ khối

A. Nguyên lý hoạt động

OPC UA hoạt động theo mô hình Client - Server.

OPC UA Server (Đóng vai trò trung tâm): Chạy trên Pi hoặc PC, khởi tạo một không gian địa chỉ chứa các điểm dữ liệu (gọi là Nodes hoặc Tags). Các Nodes này được phân nhóm rõ ràng (ví dụ: nhiệt độ, pH, trạng thái quạt, máy bơm). Server không tự tính toán vật lý hay vẽ giao diện, nó chỉ lưu trữ và quản lý quyền Đọc/Ghi dữ liệu của các client.

Các Client kết nối vào Server:

Siemens NX MCD (Client 1): Kết nối qua cổng TCP để đọc trạng thái thiết bị nhằm vẽ chuyển động 3D, đồng thời ghi nhận các va chạm vật lý ảo (như mức nước dâng chạm phao) lên Server.

FastAPI Backend (Client 2): Kết nối vào để đọc dữ liệu cảm biến (để đẩy lên Web UI) và ghi các lệnh điều khiển nhận được từ người dùng xuống các nút cơ cấu chấp hành.

Simulator Script (Client 3 - trong chế độ mô phỏng SIL): Liên tục đọc lệnh điều khiển, tính toán phản hồi vật lý ảo (ví dụ: bật quạt thì nhiệt độ giảm) rồi ghi ngược lại các thông số môi trường ảo vào các Node cảm biến trên OPC UA Server.

B. Sơ đồ khối kiến trúc truyền thông (chế độ Software-in-the-Loop - SIL)

mermaid

graph TD

subgraph "Giao diện & Điều phối"

Web["Web App Dashboard
(Giao diện điều khiển)"] <-->|WebSocket / REST| FastAPI["FastAPI Backend
(Client OPC UA)"]

end

subgraph "Bộ lưu trữ trung tâm"

OPCUA["Python OPC UA Server
(asyncua trên Port 4840)"]

end

subgraph "Giả lập & Mô phỏng Vật lý"

Sim["Python Simulator Logic
(Tính toán môi trường ảo)"] <-->|Đọc lệnh & Ghi cảm biến| OPCUA

NX["Siemens NX MCD 3D
(Client OPC UA)"] <-->|Đọc trạng thái & Vẽ chuyển động| OPCUA

end

FastAPI <-->|Đọc / Ghi các Node dữ liệu| OPCUA

3. Các đầu mục công việc chính thực hiện với OPC UA

Xây dựng OPC UA Server: Sử dụng thư viện asyncua trong Python để khởi tạo Server chạy tại cổng mặc định 4840.

Định nghĩa Cấu trúc cây thư mục (Address Space Design): Tạo các Node cha và Node con đại diện cho cảm biến, thiết bị và AI.

Lập trình script giả lập vật lý (Simulator): Tạo thuật toán tự động tính toán các biến số môi trường ảo tương thích với lệnh bật/tắt thiết bị.

Ánh xạ tín hiệu phía Siemens NX MCD (Signal Mapping): Cấu hình liên kết các biến nội bộ của mô hình 3D với các Node tương ứng trên Server.

4. Chi tiết từng nội dung thực hiện

A. Thiết kế cấu trúc các Node dữ liệu trên Server

Các Node được định danh bằng Node ID cụ thể để các Client dễ dàng truy cập:

Thư mục Cảm biến (Sensors):

ns=2;s=sensor.temp (Kiểu Float): Nhiệt độ không khí ảo.

ns=2;s=sensor.ph (Kiểu Float): Chỉ số pH trong bể chứa.

ns=2;s=sensor.water_level_1..4 (Kiểu Boolean): 4 mức nước ảo.

Thư mục Thiết bị (Actuators):

ns=2;s=actuator.fan_1..4 (Kiểu Int từ 0-255): Tốc độ quạt điều khiển bằng xung PWM.

ns=2;s=actuator.pump_220v (Kiểu Boolean): Trạng thái đóng cắt máy bơm tuần hoàn chính.

Thư mục Trí tuệ nhân tạo (AI Predictions):

ns=2;s=ai.growth_percent (Kiểu Float): Phần trăm trưởng thành của cây.

B. Cách thức đồng bộ dữ liệu thời gian thực (Data Synchronization)

Từ Web UI đến NX MCD:

Người dùng nhấn nút "Bật quạt" trên Web UI.

FastAPI nhận lệnh và ghi giá trị 255 vào Node ns=2;s=actuator.fan_1.

NX MCD lập tức nhận được sự thay đổi giá trị từ Node này và kích hoạt khớp xoay (Hinged Joint) làm cánh quạt 3D trên màn hình máy tính bắt đầu quay.

Từ NX MCD / Simulator đến Web UI:

Cảm biến va chạm ảo (Collision Sensor) trong NX MCD phát hiện mực nước ảo dâng đầy.

NX MCD ghi giá trị True vào Node ns=2;s=sensor.water_level_4.

FastAPI đọc được sự thay đổi này và đẩy qua WebSocket lên Web UI, làm đèn báo mức nước đầy trên giao diện sáng lên.

1. Giới thiệu ngắn gọn về mô hình CNN trong đồ án

CNN (Convolutional Neural Network - Mạng nơ-ron tích chập) là kiến trúc học sâu chuyên dùng cho các bài toán xử lý ảnh. Trong đồ án này, bạn sử dụng mô hình **MobileNetV2** (một nhánh cải tiến của CNN được tối ưu hóa đặc biệt cho thiết bị nhúng và di động nhờ cấu trúc tích chập tách kênh theo chiều sâu - *Depthwise Separable Convolution*) nhằm nhận diện và phân loại các loại bệnh trên lá cây xà lách từ ảnh chụp thực tế.

2. Sơ đồ luồng xử lý ảnh của mô hình CNN

mermaid

graph TD

```
User["Camera điện thoại<br>(Chụp cận cảnh lá bệnh)"] -->|Tải lên qua Web UI| API["FastAPI Backend<br>(/api/ai/disease-detect)"]
```

```
subgraph "Xử lý AI trên Raspberry Pi"
```

```
API --> Pre["Tiền xử lý ảnh<br>(Resize về 224x224, Chuẩn hóa 0-1)"]
```

```
Pre --> TFLite["Nạp ma trận ảnh vào<br>TFLite Interpreter"]
```

```
TFLite --> Infer["Chạy suy diễn CNN<br>(MobileNetV2 Model)"]
```

```
Infer --> Classify["Phân loại nhãn bệnh<br>(Softmax Layer)"]
```

```
end
```

```
Classify --> Output["Trả về kết quả JSON<br>(Tên bệnh + Độ tin cậy %)"]
```

```
Output --> DB["Lưu lịch sử vào InfluxDB"]
```

```
Output --> Web["Hiển thị kết quả chẩn đoán trên giao diện"]
```

3. Các đầu mục công việc chính thực hiện với CNN

Thu thập và gán nhãn tập dữ liệu (Dataset preparation): Chuẩn bị tập ảnh lá xà lách sạch và lá bị các bệnh phổ biến (như thối nhũn, đốm lá, cháy lá, thiếu chất...).

Huấn luyện mô hình (Training): Sử dụng Google Colab để train mạng CNN MobileNetV2 với Transfer Learning (học chuyển giao) bằng thư viện Keras/TensorFlow.

Tối ưu hóa và chuyển đổi (Model Conversion): Chuyển đổi mô hình đã huấn luyện sang định dạng TensorFlow Lite (`.tflite`) siêu nhẹ.

Triển khai trên Pi (Deployment): Viết module Python trên FastAPI để nạp file `.tflite` và thực hiện suy diễn (inference) bất đồng bộ.

4. Chi tiết từng nội dung thực hiện

A. Thu thập Dataset & Huấn luyện (Google Colab)

Tập dữ liệu: Tập hợp khoảng 500 - 1000 ảnh cho mỗi lớp (Class). Các lớp dự kiến gồm: *Healthy* (Khỏe mạnh), *Leaf Spot* (Đốm lá), *Downy Mildew* (Sương mai), *Nutrient Deficiency* (Thiếu dinh dưỡng).

Huấn luyện:

Sử dụng base model là `MobileNetV2` đã được pre-train trên tập ImageNet để tận dụng khả năng trích xuất đặc trưng của nó.

Đóng băng các layer đầu, chỉ huấn luyện các lớp phân loại phía cuối (Fine-tuning) với tập dữ liệu lá xà lách của bạn để rút ngắn thời gian huấn luyện và tăng độ chính xác.

B. Chuyển đổi sang TensorFlow Lite (TFLite)

Mô hình ban đầu sau khi train có dung lượng rất lớn (~50MB - 100MB), không tối ưu khi chạy trên CPU của Raspberry Pi 4.

Bạn sử dụng bộ công cụ `TFLiteConverter` để nén và lượng tử hóa (Quantization) mô hình từ dạng số thực dấu phẩy động 32-bit sang số nguyên 8-bit. Việc này giúp giảm dung lượng file mô hình xuống chỉ còn khoảng **dưới 10MB**, tăng tốc độ suy diễn lên gấp nhiều lần mà hầu như không làm giảm độ chính xác.

C. Triển khai suy diễn trên FastAPI

Khi có yêu cầu upload ảnh gửi tới Endpoint `POST /api/ai/disease-detect`, FastAPI sẽ thực hiện các bước:

Tiền xử lý: Chuyển đổi định dạng ảnh nhận được từ người dùng sang kích thước 224×224 pixel, chuẩn hóa các giá trị pixel về đoạn $[0, 1]$ (hoặc $[-1, 1]$ tùy cấu hình model).

Chạy suy diễn: Đưa ma trận ảnh vào `tf.lite.Interpreter`. Bộ thông dịch sẽ chạy tính toán các phép toán tích chập và trả về một mảng xác suất cho các lớp bệnh.

Hậu xử lý: Tìm nhãn lớp có xác suất cao nhất, đóng gói kết quả thành JSON trả về cho frontend và lưu lại mốc thời gian chẩn đoán vào InfluxDB.

1. Giới thiệu ngắn gọn về XGBoost

XGBoost (Extreme Gradient Boosting) là một thuật toán học máy nâng cao dựa trên cấu trúc cây quyết định (Decision Trees), sử dụng kỹ thuật nâng cấp gradient (Gradient Boosting) để tối ưu hóa hiệu năng và tốc độ tính toán. Trong đề án này, XGBoost được áp dụng dưới dạng mô hình hồi quy (Regression) nhằm dự đoán tiến trình sinh trưởng và ước tính số ngày còn lại đến kỳ thu hoạch của rau dựa trên các thông số môi trường tích lũy thu thập từ cảm biến.

2. Sơ đồ khối luồng xử lý và dự báo của XGBoost

mermaid

graph TD

```
Data["Dữ liệu cảm biến từ InfluxDB<br>(Nhiệt độ, pH, TDS, Ánh sáng, Tuổi cây)"] -->|Trích xuất đặc trưng| Features["Bộ thuộc tính đầu vào<br>(Nhiệt độ trung bình, DLI, pH trung bình)"]
```

```
Features --> XGB["Mô hình hồi quy XGBoost Regressor"]
```

```
subgraph "Nguyên lý Học suy giảm sai số (Gradient Boosting)"
```

```
XGB --> Tree1["Cây quyết định 1<br>(Đưa ra dự đoán sơ bộ)"]
```

```
Tree1 --> Residual1["Tính sai số của Cây 1<br>(Residuals)"]
```

```
Residual1 --> Tree2["Cây quyết định 2<br>(Huấn luyện để sửa sai cho Cây 1)"]
```

```
Tree2 --> Residual2["Tính sai số lũy kế"]
```

```
Residual2 --> TreeN["Cây quyết định N<br>(Tối ưu hóa hàm mất mát)"]
```

```
end
```

```
TreeN --> Aggregate["Cộng dồn dự đoán từ các cây<br>(Cộng kèm hệ số học tập Learning Rate)"]
```

```
Aggregate --> Predicts["Kết quả:<br>% Sinh trưởng & Ngày còn lại"]
```

```
Predicts --> FastAPI["FastAPI cập nhật các Node OPC UA"]
```

```
FastAPI --> NX["NX MCD tự động scale kích thước cây 3D"]
```

3. Các đầu mục công việc chính thực hiện với XGBoost

Thu thập dữ liệu huấn luyện (Data Collection): Ghi chép và tổng hợp dữ liệu môi trường (nhiệt độ, pH, TDS, ánh sáng) song song với việc đo đặc kích thước/khối lượng thực tế của cây rau qua từng ngày.

Trích xuất đặc trưng (Feature Engineering): Chuyển đổi dữ liệu thô từ cảm biến thành các thuộc tính có ý nghĩa sinh học (ví dụ: Tích lũy ánh sáng hàng ngày - DLI, độ lệch pH...).

Huấn luyện thuật toán (Model Training): Xây dựng và tinh chỉnh mô hình XGBoost Regressor bằng thư viện `xgboost` trong Python.

Triển khai dự báo tự động (Inference Deployment): Tích hợp mô hình vào FastAPI để tự động chạy dự báo định kỳ và ghi kết quả lên OPC UA Server.

4. Chi tiết nguyên lý hoạt động của thuật toán XGBoost trong đề án

A. Giải thích thuật toán Boosting & Gradient Descent

Cơ chế hoạt động: Thay vì tạo ra một cây quyết định rất lớn và phức tạp (dễ bị quá khớp - *overfitting*), XGBoost xây dựng chuỗi các cây quyết định rất đơn giản (cây yếu - *weak learners*) một cách tuần tự.

Sửa sai bằng Gradient: Cây thứ hai được xây dựng không phải để dự đoán trực tiếp mục tiêu ban đầu, mà để dự đoán **sai số (residuals)** của cây thứ nhất. Cây thứ ba sẽ dự đoán sai số của cây thứ hai, cứ thế tiếp diễn. Quá trình này tối thiểu hóa hàm mất mát (Loss Function) bằng cách sử dụng thuật toán Gradient Descent (Độ dốc giảm dần).

Công thức dự đoán cuối cùng: $\hat{y} = \sum_{k=1}^K f_k(X)$ Trong đó: $\hat{y}$ là giá trị dự đoán cuối cùng (% sinh trưởng hoặc ngày thu hoạch), $f_k(X)$ là kết quả dự đoán của cây thứ k nhân với hệ số học tập (Learning Rate) để tránh mô hình học quá nhanh.

Ưu điểm của XGBoost: Hỗ trợ cơ chế phạt (Regularization L1/L2) để kiểm soát độ phức tạp của cây, tránh hiện tượng quá khớp và có tốc độ huấn luyện cực nhanh nhờ khả năng tính toán song song.

B. Các biến đầu vào (Features) & Đầu ra (Targets) của mô hình

Mô hình XGBoost của bạn sẽ sử dụng các dữ liệu sau:

Đầu vào (Features):

cummulative_days: Số ngày cây đã được trồng trên máng (Tuổi cây).

mean_temp: Nhiệt độ không khí trung bình trong ngày.

daily_light_integral (DLI): Tổng lượng ánh sáng tích lũy mà cây nhận được mỗi ngày (tính từ cảm biến BH1750).

average_tds: Chỉ số TDS trung bình (nồng độ dinh dưỡng).

average_ph: Chỉ số pH trung bình của nước.

Đầu ra (Targets):

growth_percent (0.0 đến 100.0%): Tiến trình sinh trưởng của cây.

days_remaining (Ví dụ: từ 30 ngày đếm ngược về 0 ngày): Số ngày dự kiến còn lại để thu hoạch.

C. Triển khai đồng bộ lên Digital Twin

Hàng ngày, FastAPI gọi mô hình XGBoost để chạy dự đoán.

Kết quả dự báo được ghi trực tiếp vào các Node OPC UA: **ai.growth_percent** và **ai.days_remaining**.

Phần mềm **Siemens NX MCD** đọc giá trị **% sinh trưởng** này và gán trực tiếp vào tham số **Scale Factor** (tỷ lệ hình học) của cụm cây xà lách ảo 3D. Nhờ đó, người vận hành sẽ thấy cây ảo trên màn hình 3D lớn dần lên một cách trực quan tương quan với thuật toán AI.

1. Giới thiệu ngắn gọn về Digital Twin 3D (Siemens NX MCD)

Digital Twin 3D (Mô hình bản sao số 3D) là sự kết hợp giữa mô hình hình học 3D thiết kế trên phần mềm CAD và các thuật toán mô phỏng vật lý, động học cơ điện tử của module **Siemens NX MCD (Mechatronics Concept Designer)**. Bản sao số này kết nối trực tiếp với hệ thống điều khiển thực thông qua OPC UA để hoạt động và phản hồi động học thời gian thực giống hệt như một giàn cây vật lý.

2. Sơ đồ luồng tương tác vật lý của Digital Twin 3D

mermaid

graph LR

```
subgraph "Mô phỏng Cơ điện tử trên Siemens NX MCD"
    Rigid["Rigid Body<br>(Khai báo khối vật lý)"] --> Joints["Joints & Motion<br>(Khớp quay quạt, khớp tịnh tiến phao)"]
    Physics["Physics Simulation<br>(Hạt nước chảy máng NFT, trọng lực)"]
    Sensors3D["Collision Sensors<br>(Cảm biến va chạm đo mực nước)"]
end

subgraph "Trao đổi tín hiệu thời gian thực (Port 4840)"
    OPC["OPC UA Server Nodes"]
end

subgraph "Hệ thống điều khiển & Web"
    Ctrl["FastAPI / Web App / Simulator"]
end

Ctrl -->|Ghi lệnh điều khiển: PWM quạt/bơm, Relay đèn| OPC
OPC -->|Đọc lệnh kích hoạt chuyển động| Joints
OPC -->|Đọc % sinh trưởng chỉnh Scale| Rigid
Sensors3D -->|Ghi tín hiệu mực nước đạt ngưỡng| OPC
OPC -->|Đọc dữ liệu hiển thị màn hình và Web UI| Ctrl
```

3. Các đầu mục công việc chính thực hiện với Digital Twin 3D

Thiết kế mô hình 3D CAD: Vẽ mô hình tỷ lệ 1:1 của khung sắt, máng NFT, bể nước, đèn LED, cánh quạt và mô hình cây xà lách.

Thiết lập thuộc tính cơ điện tử (MCD Physics): Định nghĩa các đối tượng vật lý cứng (Rigid Body), khớp động học (Hinged/Prismatic Joint) và các hạt chất lỏng (Water Particles).

Cấu hình kết nối OPC UA: Thiết lập liên kết tín hiệu (External Signal Configuration) giữa NX MCD và OPC UA Server.

Ánh xạ tín hiệu và chạy mô phỏng: Kết nối các tín hiệu điều khiển vào cơ cấu chấp hành ảo (quạt xoay, bơm phun nước) và kiểm tra hoạt động Software-in-the-Loop.

4. Chi tiết các hoạt động mô phỏng cơ điện tử trong NX MCD

A. Mô phỏng động học quạt thông gió và máy bơm

Quạt thông gió: Cánh quạt được gắn khớp xoay (**Hinged Joint**) kết hợp với cảm biến tốc độ (**Speed Control**). Khi nhận tín hiệu PWM (0-255) từ OPC UA, giá trị này sẽ được nhân với một hệ số để chuyển thành tốc độ vòng/phút ảo, làm cánh quạt 3D quay nhanh hay chậm trực quan trên màn hình.

Máy bơm nước: Sử dụng tính năng tạo hạt vật lý (**Liquid/Particle Path**) để giả lập dòng nước chảy. Lưu lượng nước phun ra từ vòi cấp vào máng NFT sẽ tăng giảm tỷ lệ thuận theo giá trị PWM của bơm nhận được từ OPC UA. Dưới tác dụng của trọng lực ảo, các hạt nước sẽ tự động chạy dọc máng NFT và rơi ngược về bể chứa.

B. Mô phỏng mực nước và cảm biến tiệm cận ảo

Phao đo nước: Phao cơ trong bể được thiết lập khớp tịnh tiến (**Prismatic Joint**) để di chuyển lên xuống tự do theo mực nước ảo trong bể chứa.

Cảm biến mực nước tiệm cận (giả lập Y26): Đặt các cảm biến va chạm ảo (**Collision Sensor**) tại 4 cao độ khác nhau trong bể. Khi nước dâng lên hoặc phao ảo di chuyển chạm vào vùng cảm biến, tín hiệu logic tương ứng sẽ lập tức được ghi nhận và gửi ngược lại OPC UA Server để báo về hệ thống.

C. Mô phỏng ánh sáng đèn LED Grow

Thiết lập thuộc tính vật liệu phát quang (**Emission/Luminance**) cho cụm mô hình đèn LED Grow.

Khi nhận tín hiệu Relay **ON** từ OPC UA, vật liệu của đèn sẽ tự động thay đổi từ màu xám tối sang màu hồng/đỏ phát sáng rực rỡ, biểu thị đèn đang hoạt động chiếu sáng bổ sung quang phổ cho rau.

D. Trực quan hóa tiến trình sinh trưởng của cây (Scale Factor)

Để mô phỏng quá trình lớn lên của cây trong vòng vài chục ngày chỉ trong vài phút mô phỏng, tham số tỷ lệ (**Scale Factor**) của cụm cây xà lách 3D được liên kết trực tiếp với Node **ai.growth_percent** từ OPC UA.

Khi giá trị phần trăm tăng dần từ 0% lên 100%, cụm mô hình cây ảo sẽ tự động phóng to đều theo cả 3 chiều trục vật lý, giúp quan sát trực quan sự phát triển của cây tương quan với dữ liệu thu thập.
